

Skill Challenge #16



#multipleinheritance

Requirements

- > Define a new type called **CreditCard** that supports a `generate()` method that randomly chooses the middle 14 digits of a credit card. This type should also support a `number` property that reflects the generated number is spaced blocks of 4 digits, similar to how they would show on a credit card, e.g. 5241 0213 8828 6423
- > Then, define two other types that are not meant to be instantiated but rather only cooperatively work with the `generate()` method from `CreditCard`:
 - **VisaMixin**, which prepends the digits '42', and
 - **MasterCardMixin**, which prepends the digits '53'
- > Then define another type **ValidMixin** similar to the above that will be responsible for putting in the right checksum. To calculate the checksum, we could use Luhn's algorithm:
 - keep track of a cumulative sum, initially 0
 - starting from the rightmost digit (i.e. 15th), work leftward
 - find the double ($2 * n$) of every other number, starting with the one above
 - if the double is greater than 9, add the sum of its digits to the cumulative sum (e.g. for 12 -> add $1+2$ -> 3)
 - if not, add the double ($2 * n$)
 - for other non-doubled numbers, add the number itself
 - in the end, return the difference between 10 and cumulative sum mod 10
- > The integer returned by the above becomes the 16th digit of the credit card if and only if the `ValidMixin` for valid credit cards is included in a class definition.

